

Presentation Evidence Capture Guide

이 문서는 발표 슬라이드에서 "정말 구현했는가?"라는 의심을 해소하기 위한 캡처 대상을 정리합니다.

목표는 기능을 예쁘게 소개하는 것이 아니라, 프론트엔드, 백엔드, 파이프라인, 운영 DB, 데이터웨어하우스가 실제로 구현되어 실행되고 있음을 보여주는 것입니다. 따라서 화면 캡처뿐 아니라 코드, API 문서, Docker 실행 상태, DB row, ELT 로그를 함께 보여주는 구성이 좋습니다.

Recommended Capture Set

최종 슬라이드에 넣을 캡처는 아래 순서를 추천합니다.

1. Docker 실행 상태: `docker compose ps`
2. Frontend 화면: 학생/교사 대시보드
3. Backend API: FastAPI Swagger /docs
4. Pipeline: 분석 코드 또는 분석 품질 화면
5. Operational DB: MySQL 테이블과 `metrics` row
6. PII Encryption: MySQL의 `enc:v1:...` 암호문 row
7. Data Warehouse: PostgreSQL `raw/stg/mart` 테이블
8. ELT 실행 결과: Validation PASS 또는 `elt_run_history`

1. 전체 Docker 실행 증거

Capture Target

터미널에서 다음 명령 결과를 캡처합니다.

```
docker compose ps
```

캡처에 보이면 좋은 항목:

```
aic_frontend    Up
aic_backend     Up
aic_pipeline    Up
aic_db          Up / healthy
aic_warehouse   Up / healthy
```

Slide Message

프론트엔드, 백엔드, 분석 파이프라인, 운영 DB, 데이터웨어하우스를 Docker 기반으로 분리 실행

Why This Works

이 캡처는 "서비스를 실제로 띄웠는가?"에 대한 가장 직접적인 증거입니다. 발표 초반에 배치하면 이후 설명의 신뢰도가 올라갑니다.

2. Frontend 구현 증거

Capture Target

브라우저에서 실제 화면을 캡처합니다. URL이 보이도록 캡처하면 더 좋습니다.

추천 화면:

```
/student/dashboard  
/teacher/dashboard  
/teacher/students  
/teacher/advanced  
/teacher/analytics/assignment/1
```

함께 넣기 좋은 보조 캡처:

```
aic-frontend/src/router/index.js  
aic-frontend/src/api/index.js  
aic-frontend/src/stores/auth.js
```

Slide Message

Vue 기반 화면, 라우팅, 상태 관리, API 연동 구현

Suggested Bullet Points

- Vue 3 + Vite 기반 SPA 구현
- Vue Router로 학생/교사/관리자 화면 분리
- Pinia store로 인증 및 사용자 상태 관리
- Axios client로 `/api/v1` backend 호출
- Chart.js 기반 AIC 지표 시각화

Why This Works

화면 캡처만 넣으면 정적 목업처럼 보일 수 있습니다. 실제 라우터 코드나 API client 캡처를 같이 넣으면 구현 증거가 됩니다.

3. Backend API 구현 증거

Capture Target

FastAPI Swagger 문서 화면을 캡처합니다.

추천 URL:

```
/docs
```

보이면 좋은 API 그룹:

```
/api/v1/auth  
/api/v1/student  
/api/v1/teacher  
/api/v1/admin  
/api/v1/jobs
```

가능하면 API 하나를 펼쳐 request/response schema가 보이게 캡처합니다.

함께 넣기 좋은 보조 캡처:

```
aic-backend/app/routers/teacher.py  
aic-backend/app/routers/student.py  
aic-backend/app/services/teacher_service.py  
aic-backend/app/dependencies.py
```

Slide Message

FastAPI 기반 REST API, 인증/권한, 학생/교사 기능 구현

Suggested Bullet Points

- FastAPI 기반 API 서버 구현
- JWT 인증 및 역할 기반 접근 제어
- 학생/교사/관리자 API 분리
- SQLAlchemy AsyncSession 기반 MySQL 비동기 접근
- Pipeline 호출 및 분석 job 상태 관리
- 개인정보 필드 암호화 적용

Why This Works

Swagger는 실제 서버에 등록된 API 목록을 보여줍니다. 코드 캡처보다 비개발자에게도 설득력이 좋습니다.

4. Pipeline 구현 증거

Capture Target

분석 파이프라인 코드 또는 분석 결과 화면을 캡처합니다.

추천 코드:

```
aic-pipeline/app/main.py
aic-pipeline/app/pipeline_runner.py
aic-pipeline/app/schemas.py
```

추천 화면:

```
/admin/analysis-quality
/teacher/analytics/assignment/1
/teacher/advanced
```

Slide Message

SBERT/TF-IDF 기반 분석 파이프라인과 AIC metric 계산 구현

Suggested Bullet Points

- FastAPI 기반 pipeline 서비스 구현
- SBERT / TF-IDF 기반 텍스트 분석
- PI, UI, OI, AIC score 계산
- CPU 기반 실행 구조 유지
- backend가 pipeline을 호출하고 결과를 MySQL *metrics*에 저장

Why This Works

분석 결과 화면과 pipeline 코드를 함께 보여주면 단순 차트가 아니라 실제 분석 서비스가 존재한다는 점을 증명할 수 있습니다.

5. Operational DB 구현 증거

Capture Target

DBeaver에서 MySQL *aic_db* 연결 화면을 캡처합니다.

추천 캡처:

```
aic_db 테이블 목록
users
submissions
metrics
```

```
teacher_feedback  
analysis_jobs
```

특히 보여주면 좋은 row:

```
metrics.pi_score  
metrics.ui_score  
metrics.oi_score  
metrics.aic_score  
analysis_jobs.status  
submissions.assignment_id  
submissions.student_id
```

Slide Message

운영 DB에 사용자, 과제, 제출, 분석 결과, 피드백 저장 구조 구현

Suggested Bullet Points

- MySQL 8 기반 운영 DB 구성
- 사용자, 수업, 과제, 제출, 분석 job 저장
- **metrics** 테이블에 PI/UI/OI/AIC 점수 저장
- **teacher_feedback** 테이블에 교사 피드백 저장
- backend가 저장과 권한 검사를 담당

Why This Works

DB row는 "화면만 만든 것인가?"라는 의심을 줄이는 데 강합니다. 특히 **metrics** row는 분석 결과가 실제로 저장되었음을 보여줍니다.

6. PII Encryption 구현 증거

Capture Target

DBeaver에서 MySQL row를 캡처합니다.

추천 테이블:

```
users  
teacher_feedback
```

보이면 좋은 값:

```
users.name = enc:v1:...
users.email = enc:v1:...
teacher_feedback.content = enc:v1:...
```

함께 넣기 좋은 보조 캡처:

```
aic-backend/app/security.py
aic-backend/app/models/db_models.py
aic-backend/app/config.py
```

Slide Message

이름, 이메일, 피드백 필드는 backend 경계에서 암호화하고 DB에는 암호문으로 저장

Suggested Bullet Points

- **ENCRYPTION_KEY** 환경변수 기반 필드 암호화
- 저장 시 **enc:v1:...** 형태로 암호화
- API 응답 시 backend에서 복호화
- frontend는 기존 API 사용 방식 유지
- warehouse에는 암호문 그대로 적재

Why This Works

보안 기능은 말로 설명하면 추상적으로 들립니다. DB에 실제 암호문이 저장된 row와 backend 암호화 코드 캡처를 같이 보여주는 것이 가장 좋습니다.

7. Data Warehouse 구현 증거

Capture Target

DBeaver에서 PostgreSQL **aic_warehouse** 연결 화면을 캡처합니다.

추천 테이블:

```
raw_users
raw_classes
raw_assignments
raw_submissions
raw_metrics
stg_submission_metrics
mart_student_assignment_metrics
mart_assignment_summary
```

```
mart_class_summary
elt_run_history
```

가장 좋은 캡처 구성:

왼쪽: warehouse 테이블 목록
오른쪽: mart_assignment_summary 또는 mart_class_summary 조회 결과
아래: elt_run_history 성공 row

Slide Message

운영 DB 데이터를 warehouse로 적재하고, 분석용 mart를 생성

Suggested Bullet Points

- PostgreSQL 기반 데이터웨어하우스 구성
- 운영 DB의 핵심 테이블을 raw layer로 적재
- stg_submission_metrics에서 제출, 학생, 과제, 점수 결합
- 학생/과제/수업 단위 mart 생성
- elt_run_history로 적재 결과 기록

Why This Works

raw -> stg -> mart 테이블이 실제 DB에 존재하고 row가 조회되는 장면은 데이터 파이프라인 구현 증거로 매우 강합니다.

8. ELT 실행 증거

Capture Target

터미널에서 다음 명령 실행 결과를 캡처합니다.

```
docker compose run --rm elt
```

보이면 좋은 로그:

```
ELT run summary
Validation: PASS
ELT completed successfully.
```

또는 DBeaver에서 elt_run_history 테이블의 성공 row를 캡처합니다.

Slide Message

운영 DB에서 warehouse까지 ELT 적재 및 검증 자동화

Suggested Bullet Points

- MySQL source table 추출
- PostgreSQL raw layer upsert
- staging/mart 변환
- row count 및 mart count 검증
- 실행 이력 저장

Why This Works

warehouse 테이블만 보여주면 수동으로 만든 것처럼 보일 수 있습니다. ELT 성공 로그 또는 `elt_run_history`를 함께 보여주면 자동 적재 흐름을 증명할 수 있습니다.

Suggested Slide Layout

참고 슬라이드와 비슷한 스타일로 구성하려면 한 장에 너무 많은 설명을 넣기보다, 캡처 중심으로 배치합니다.

상단: 슬라이드 제목
왼쪽: 실제 화면 또는 DB 캡처
가운데: 코드/API/실행 로그 캡처
오른쪽: 구현 완료 요약 박스
하단: 다음 개선 방향 또는 검증 포인트

Recommended Slide Order

1. 전체 실행 구조와 Docker 실행 상태
2. Frontend 구현 증거
3. Backend API 구현 증거
4. Pipeline 분석 구현 증거
5. Operational DB 구현 증거
6. 개인정보 암호화 구현 증거
7. Data Warehouse 구현 증거
8. ELT 실행 및 검증 증거

One-Slide Summary Option

발표 시간이 짧다면 한 장에 다음 네 가지를 넣습니다.

1. `docker compose ps`
2. `FastAPI Swagger /docs`

3. MySQL metrics + encrypted PII row
4. PostgreSQL mart + elt_run_history

이 조합은 UI보다 구현 증거에 더 집중합니다. "화면만 만든 것인가?"라는 의심을 해소하려면 API, DB, Docker, ELT가 함께 보이는 것이 가장 효과적입니다.